Design of a LED controller

Name		**Class**	
Date		**Score**	
Examiner			

Contents

1 Introduction **3**

2 Experiment procedure **3**

2.1 Tools . . . 3

2.2 The Project Overview . . . 4

2.3 LED control design . . . 5

2.4 Testbench Design . . . 9

2.5 Programming the FPGA . . . 12

2.6 Waveform . . . 14

2.7 FPGA experiment . . . 15

3 Analysis and discussion **15**

3.1 System Design and Functionality . . . 15

3.1.1 Reset Functionality . . . 15

3.1.2 Button-Controlled LED Patterns . . . 16

3.1.3 Timing Control . . . 16

3.2 Testbench Design and Simulation . . . 16

3.2.1 Reset Test . . . 16

3.2.2 Button Press Tests . . . 17

3.2.3 No Button Press Test . . . 17

3.2.4 Timing Verification . . . 17

3.2.5 Simulation Results . . . 17

3.3 FPGA Programming and Testing . . . 17

3.3.1 Pin Assignment . . . 17

3.4 Challenges and Solutions . . . 18

3.4.1 Timing Issues . . . 18

3.4.2 Button Debouncing . . . 18

3.4.3 Testbench Verification . . . 18

4 Conclusion and self-review **18**

1 Introduction

This experiment involves designing and implementing a digital LED controller using VHDL. The system is designed to control multiple LEDs through push-button inputs using an FPGA platform. The design employs state machines and counter logic to manage different LED patterns based on user input, demonstrating fundamental concepts in digital system design.

2 Experiment procedure

2.1 Tools

(1) Quartus II 13.0

(2) ModelSim

(3) Computer

(4) FPGA (clocking at 50MHz)

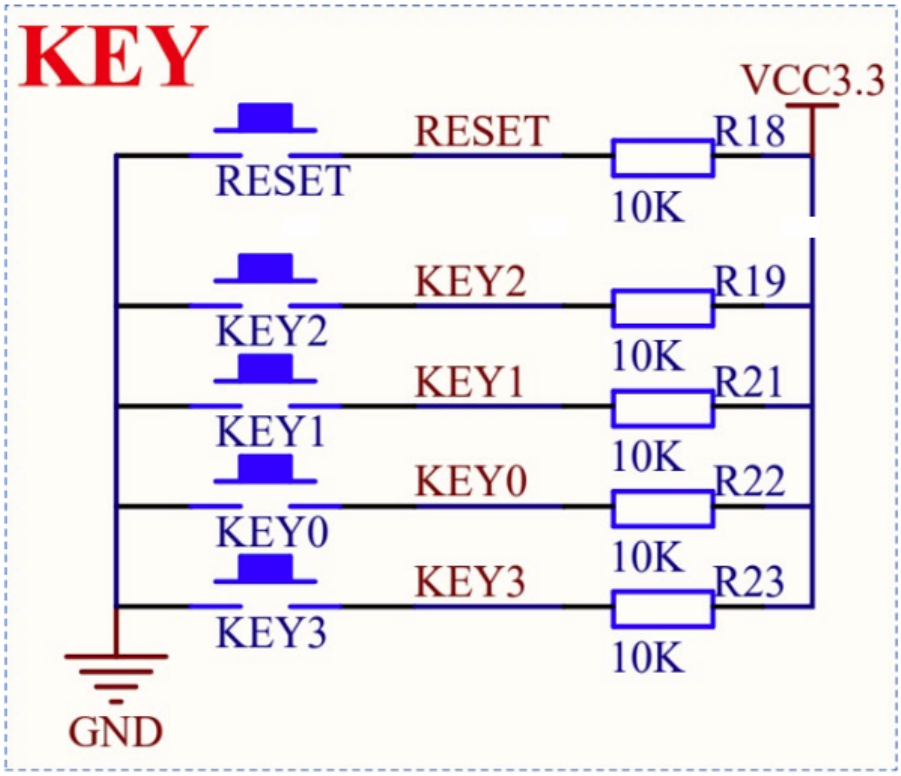

Figure 1: System design

2.2 The Project Overview

(1) Design the VHDL code to describe the LED controller. The controller has 4 control buttons and a reset button.

(2) Design a testbench to test the function of the control system, by pressing the reset button and 4 control buttons, respectively

(3) Program the VHDL code to the FPGA and test its function by pressing the corresponding buttons.

The brief workflow are shown as figure2 below

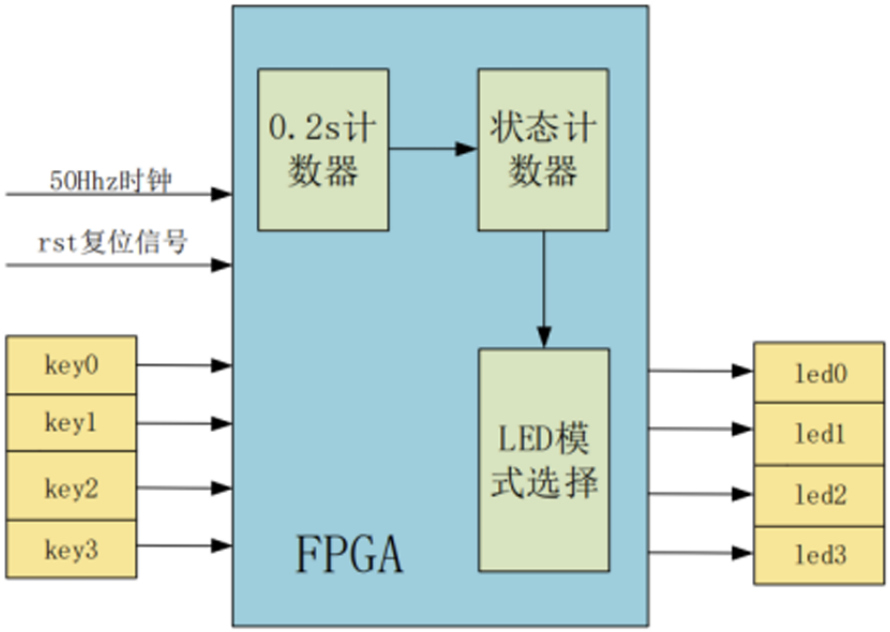

Figure 2: Workflow

2.3 LED control design

Below are our implement of the LEDs

```
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
USE IEEE.std_logic_unsigned.ALL;

entity key_ctrl is
port(sys_clk:in std_logic;
      sys_rst_n:in std_logic;
      key: in std_logic_vector(3 downto 0);
      led: out std_logic_vector(3 downto 0)
     );
```

```
end key_ctrl;

ARCHITECTURE Behavioral OF key_ctrl IS

signal cnt: std_logic_vector(23 downto 0);
signal led_control: std_logic_vector(1 downto 0);

BEGIN
process(sys_clk,sys_rst_n)
begin
if (sys_rst_n = '0') then
    cnt <= (others => '0');
elsif (rising_edge(sys_clk)) then
    if (cnt < "100110001001011001111111") then
    cnt <= cnt + 1;
    else
    cnt <= (others => '0');
    end if;
end if;

end process;
process(sys_clk,sys_rst_n)
begin
if (sys_rst_n = '0') then
    led_control <= "00";
elsif (rising_edge(sys_clk)) then
    if (cnt = "100110001001011001111111") then
    led_control <= led_control + 1;
    else
    led_control <= led_control;
    end if;
end if;
end process;

```

```
process(sys_clk, sys_rst_n)
begin
if (sys_rst_n='0') then
    led <= "0000";
elsif (rising_edge(sys_clk)) then
    if (key(0) = '0') then
        case led_control is
        when "00" =>
        led <= "1000";
        when "01" =>
        led <= "0100";
        when "10" =>
        led <= "0010";
        when "11" =>
        led <= "0001";
        when others =>
        led <= "0000";
        end case;
    elsif (key(1) = '0') then
        case led_control is
        when "00" =>
        led <= "0001";
        when "01" =>
        led <= "0010";
        when "10" =>
        led <= "0100";
        when "11" =>
        led <= "1000";
        when others =>
        led <= "0000";
        end case;
    elsif (key(2) = '0') then
        case led_control is
        when "00" =>
```

```
        led <= "1111";
        when "01" =>
        led <= "0000";
        when "10" =>
        led <= "1111";
        when "11" =>
        led <= "0000";
        when others =>
        led <= "0000";
        end case;
    elsif (key(3) = '0') then
        led <= "1111";
    else
        led <= "0000";
    end if;
end if;
end process;

END Behavioral;
```

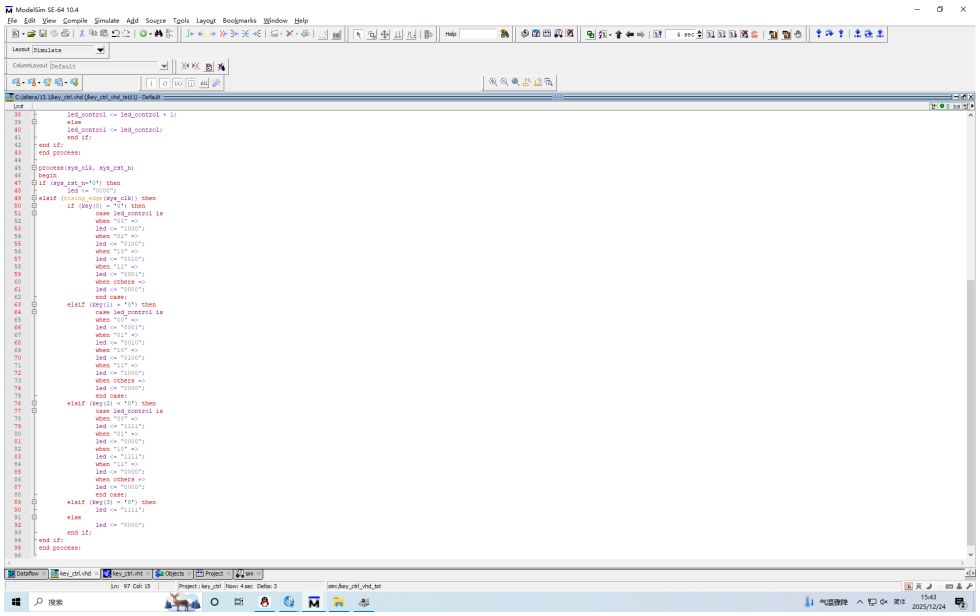

Figure 3: LED Controller Implementation

2.4 Testbench Design

Below are our implement of the testbench

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY key_ctrl_vhd_tst IS
END key_ctrl_vhd_tst;
ARCHITECTURE key_ctrl_arch OF key_ctrl_vhd_tst IS
-- constants
-- signals
SIGNAL key : STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL led : STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL sys_clk : STD_LOGIC;
SIGNAL sys_rst_n : STD_LOGIC;
CONSTANT clk_period : time := 20ns;
```

```
COMPONENT key_ctrl
    PORT (
    key : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
    led : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
    sys_clk : IN STD_LOGIC;
    sys_rst_n : IN STD_LOGIC
    );
END COMPONENT;
BEGIN
    i1 : key_ctrl
    PORT MAP (
-- list connections between master ports and signals
    key => key,
    led => led,
    sys_clk => sys_clk,
    sys_rst_n => sys_rst_n
    );
clk_gen : process
begin
    sys_clk <= '1';
    wait for clk_period/2;
    sys_clk <= '0';
    wait for clk_period/2;
end process;
init : PROCESS
-- variable declarations
BEGIN
       key <= "1111";
    sys_rst_n <= '0';
    wait for 400ms;
    sys_rst_n <= '1';
    wait for 200ms;
    key(0) <= '0';
    wait for 800ms;
```

```
    key(0) <= '1';
    key(1) <= '0';
    wait for 800ms;
    key(1) <= '1';
    key(2) <= '0';
    wait for 800ms;
    key(2) <= '1';
    key(3) <= '0';
    wait for 800ms;
    key(3) <= '1';
WAIT;
END PROCESS init;
always : PROCESS
-- optional sensitivity list
-- (        )
-- variable declarations
BEGIN
        -- code executes for every event on sensitivity list
WAIT;
END PROCESS always;
END key_ctrl_arch;
```

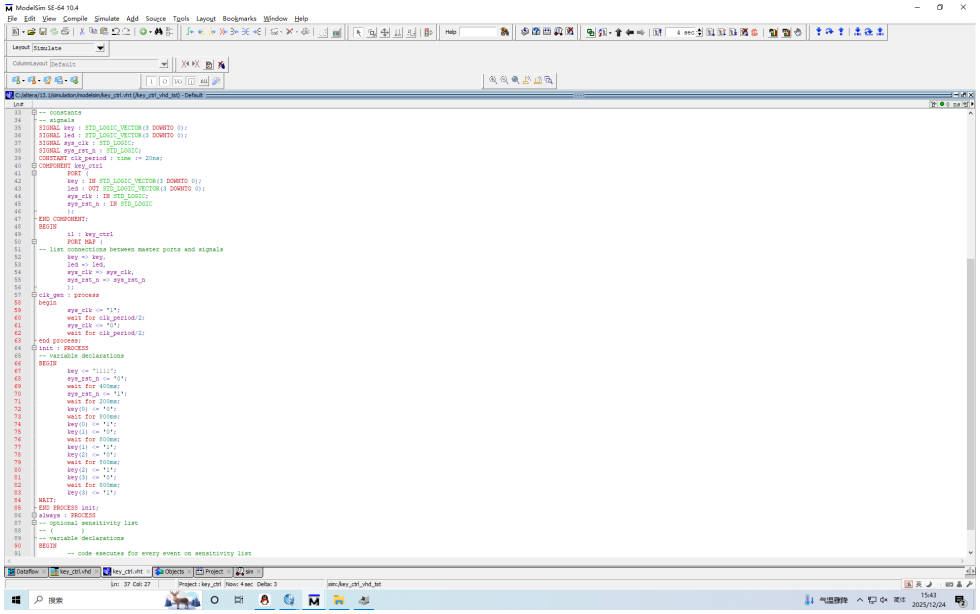

Figure 4: Testbench Implementation

2.5 Programming the FPGA

Assignment pin planner

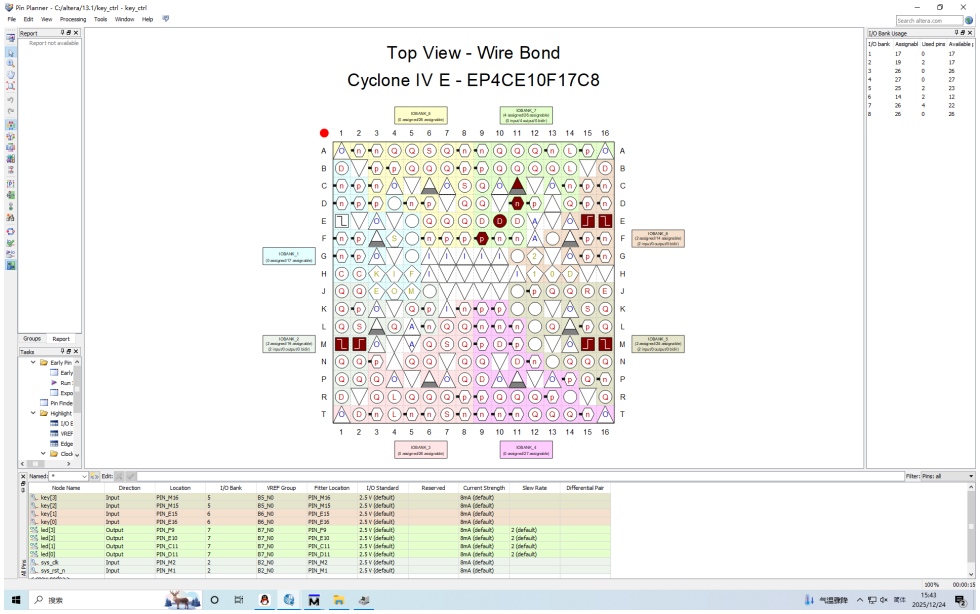

Figure 5: Pin assignment

信号名	方向	管脚	端口说明
sys_clk	input	M2	系统时钟，50M
sys_rst_n	input	M1	系统复位，低有效
key[0]	input	E16	按键
key[1]	input	E15	按键
key[2]	input	M15	按键
key[3]	input	M16	按键
led[0]	output	D11	LED
led[1]	output	C11	LED
led[2]	output	E10	LED
led[3]	output	F9	LED

Figure 6: Pin assignment

2.6 Waveform

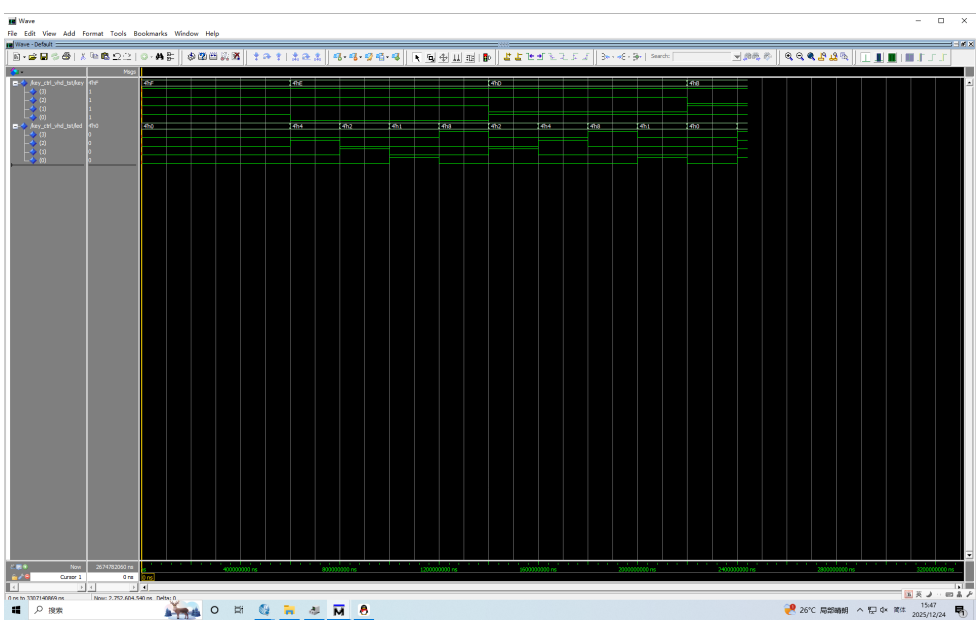

Figure 7: Simulation Of The Waveform

2.7 FPGA experiment

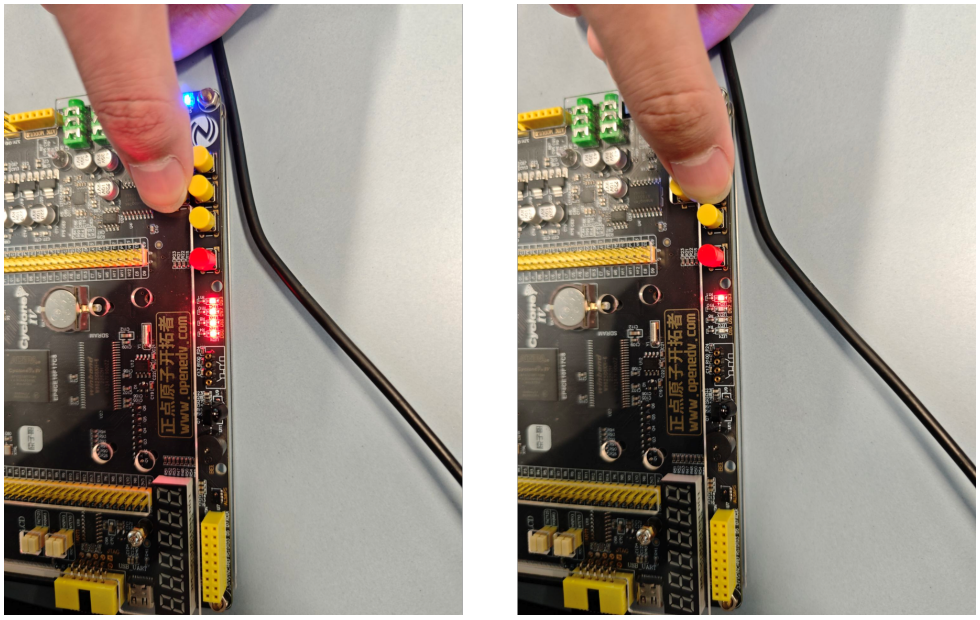

Figure 8: FPGA experiment

3 Analysis and discussion

3.1 System Design and Functionality

The system designed in this experiment is a simple yet effective digital circuit that controls four LEDs based on the state of four push-button switches. The system operates under the control of a clock signal (`sys_clk`) and a reset signal (`sys_rst_n`). The behavior of the LEDs is determined by the state of the buttons (`key`) and a counter (`cnt`) that generates a timing signal to control the LED patterns.

3.1.1 Reset Functionality

When the reset button is pressed (`sys_rst_n` = '0'), the system resets to its initial state, turning off all LEDs (`led` = "0000"). This ensures that the system starts from a known state, which is crucial for reliable operation.

3.1.2 Button-Controlled LED Patterns

- **Button key(0) (Right-to-Left Flow):** When `key(0)` is pressed, the LEDs light up in a sequence from right to left (`led(3)` to `led(0)`). This pattern is achieved by incrementing a 2-bit control signal (`led_control`) that determines the current LED state.
- **Button key(1) (Left-to-Right Flow):** When `key(1)` is pressed, the LEDs light up in a sequence from left to right (`led(0)` to `led(3)`). This pattern is also controlled by the `led_control` signal, but the sequence is reversed.
- **Button key(2) (Blinking):** When `key(2)` is pressed, the LEDs blink on and off in a repeating pattern. This effect is achieved by toggling the LEDs on and off based on the `led_control` signal.
- **Button key(3) (All LEDs On):** When `key(3)` is pressed, all LEDs are turned on (`led` = "1111"). This is a simple state where no pattern is involved, and all LEDs remain on.
- **No Button Pressed:** When no button is pressed, the LEDs remain off (`led` = "0000").

3.1.3 Timing Control

The system uses a 24-bit counter (`cnt`) to generate a timing signal. The counter increments on each rising edge of the clock (`sys_clk`) and resets to zero when it reaches a specific value ("100110001001011011111111"), which corresponds to a delay of approximately 0.2 seconds at a 50 MHz clock frequency. This timing signal is used to control the interval between LED state changes, ensuring that the patterns are displayed at the desired rate.

3.2 Testbench Design and Simulation

The testbench is designed to verify the functionality of the system by simulating the behavior of the push-button switches and observing the corresponding LED patterns. The testbench includes the following features:

3.2.1 Reset Test

The testbench first checks the reset functionality by pressing the reset button and verifying that all LEDs turn off.

3.2.2 Button Press Tests

The testbench simulates pressing each of the four buttons (`key(0)` to `key(3)`) and verifies that the LEDs display the correct patterns:

- **Button key(0) (Right-to-Left Flow):** The LEDs should light up in sequence from right to left.
- **Button key(1) (Left-to-Right Flow):** The LEDs should light up in sequence from left to right.
- **Button key(2) (Blinking):** The LEDs should blink on and off in a repeating pattern.
- **Button key(3) (All LEDs On):** All LEDs should remain on.

3.2.3 No Button Press Test

The testbench also checks the behavior when no button is pressed, ensuring that all LEDs remain off.

3.2.4 Timing Verification

The testbench verifies that the interval between LED state changes is approximately 0.2 seconds, as expected.

3.2.5 Simulation Results

The simulation results show that the system behaves as expected, with the LEDs displaying the correct patterns based on the state of the buttons. The timing of the LED patterns is also verified to be accurate.

3.3 FPGA Programming and Testing

The FPGA programming process involves several key steps:

3.3.1 Pin Assignment

The Assignment-Pin Planner is used to allocate the pins properly, ensuring that the inputs (buttons) and outputs (LEDs) are correctly mapped.

3.4 Challenges and Solutions

3.4.1 Timing Issues

One of the challenges in this experiment was ensuring that the timing of the LED patterns was accurate. The 24-bit counter was designed to generate a timing signal every 0.2 seconds, but slight variations in the clock frequency could affect the timing. To address this, the counter value was carefully calculated to ensure that the timing was within the desired range.

3.4.2 Button Debouncing

Another challenge was dealing with button debouncing, which can cause unintended state changes when a button is pressed. To mitigate this, the system was designed to ignore short-duration button presses, ensuring that only stable button presses are recognized.

3.4.3 Testbench Verification

Writing a comprehensive testbench was challenging, especially when simulating the timing of the LED patterns. The testbench was designed to include all possible inputs and verify the system's behavior under different conditions. The use of ModelSim for waveform simulation helped in visualizing the system's behavior and verifying its correctness.

4 Conclusion and self-review

The experiment successfully demonstrates the design and implementation of a digital system that controls four LEDs using four push-button switches. The system is developed in VHDL and tested using a testbench in ModelSim. The FPGA is programmed to display various LED patterns based on the state of the buttons, with its functionality verified through both simulation and physical testing. This experiment emphasizes the critical roles of timing control, button debouncing, and thorough testing in the design of digital systems. The results confirm that the system performs as expected, with the LEDs displaying the correct patterns corresponding to the button states. The use of VHDL and ModelSim provides a solid framework for designing and testing digital systems, making it an invaluable tool for future projects.